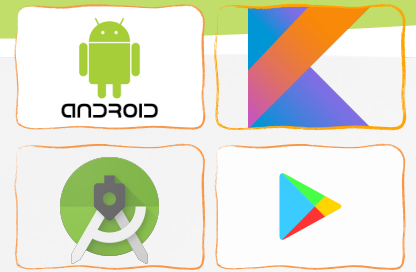




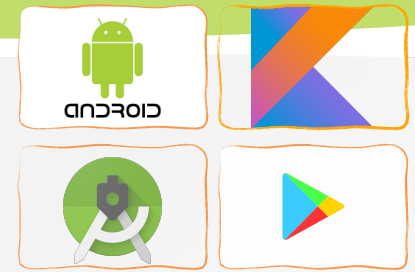
Software Development for Mobile Devices

Nguyen Le Hoang Dung
[*nlhdung@fit.hcmus.edu.vn*](mailto:nlhdung@fit.hcmus.edu.vn)



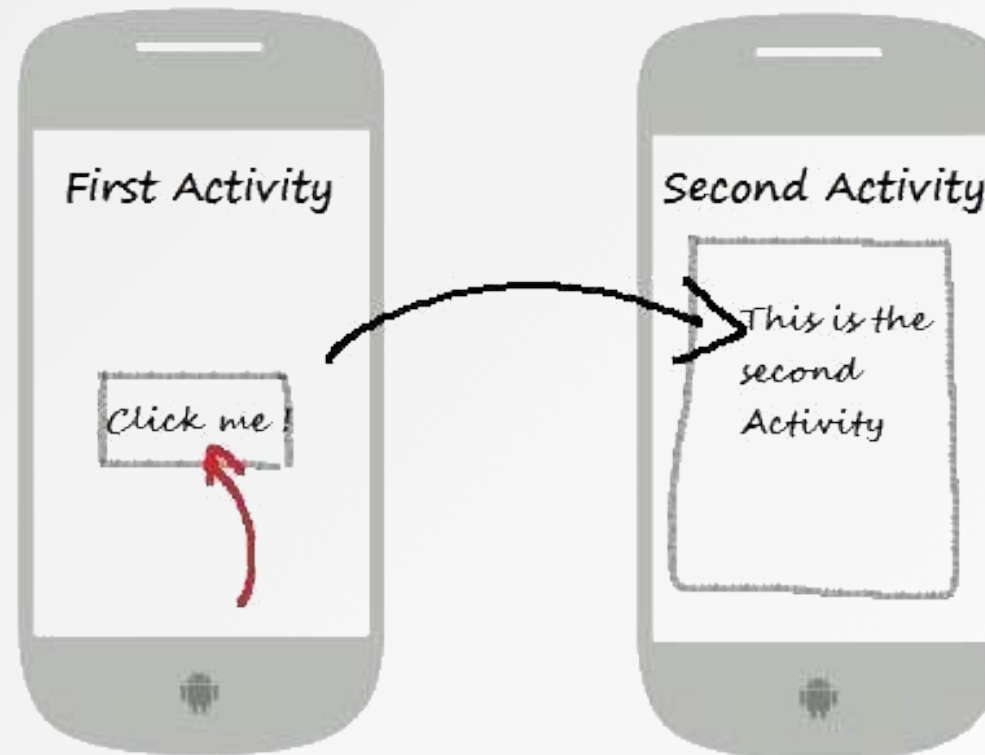
Content

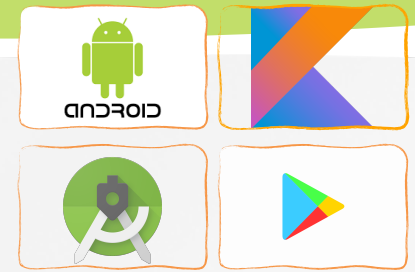
- » Activity
- » Intent
- » Fragments



About Activity

- » Most apps contain multiple screens (**multiple activities**).
- » **main activity** is a specific activity in an app.

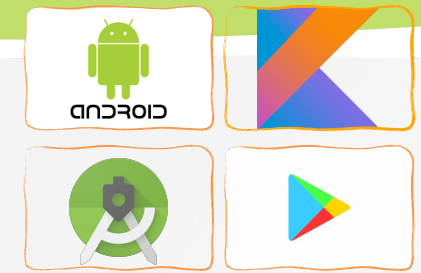




About Activity

- » Most apps contain multiple screens (**multiple activities**).
- » **main activity** is a specific activity in an app.

```
5      <application
6          android:allowBackup="true"
7          android:icon="@mipmap/ic_launcher"
8          android:label="ActivityDemo"
9          android:roundIcon="@mipmap/ic_launcher_round"
10         android:supportRtl="true"
11         android:theme="@style/AppTheme">
12         <activity android:name=".MainActivity">
13             <intent-filter>
14                 <action android:name="android.intent.action.MAIN" />
15                 <category android:name="android.intent.category.LAUNCHER" />
16             </intent-filter>
17         </activity>
18     </application>
```

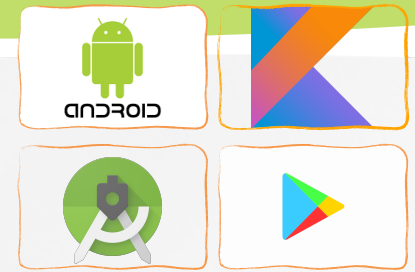



About Activity

- » Activity must be **registered** information about them in the **app's manifest**
 - Manifest file: add **<activity>** as a child of the **<application>**
 - Attribute: name, label, icon, UI theme, ...

```
<activity android:name=".MainActivity" >  
    <intent-filter>  
        <action android:name="android.intent.action.MAIN" />  
  
        <category android:name="android.intent.category.LAUNCHER" />  
    </intent-filter>  
</activity>
```

```
<activity android:name=".GreetingActivity" >  
</activity>
```

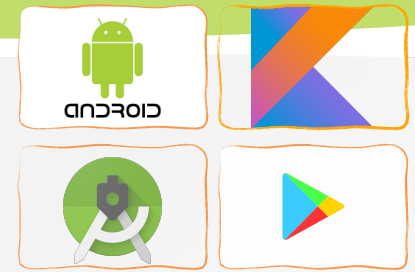


Android Intents

» An Intent is a messaging object you can use to request an action from another app component.

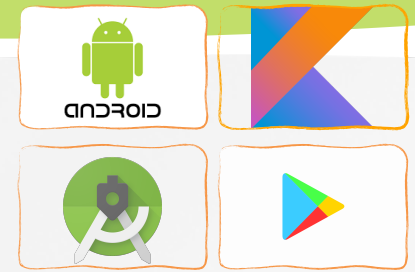
– Starting an activity

- Start a new instance of an **Activity** by passing an **Intent** to **startActivity()**. It also carries any necessary data.
- Receive a result from the activity when it finishes, call **startActivityForResult()** and **onActivityResult()** callback.



Android Intents (cont.)

- » There are two types of intents:
 - **Explicit intents** specify which application will satisfy the intent
 - Supply the target app's package name
 - or supply a fully-qualified component class name.
 - **Implicit intents**
 - Do not name a specific component.
 - Declare a general action to perform, which allows a component from another app to handle it.



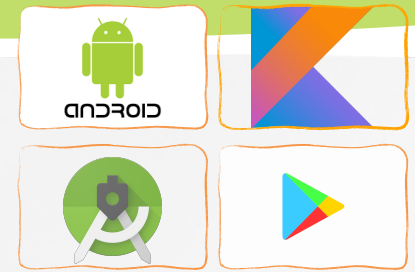
Android Intents (cont.)

» Building an intent

- An **Intent** object carries information that the Android system uses to determine which component to start

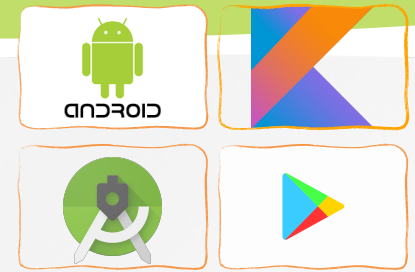
» The primary information in an intent

- **name**: the name of the component to start
- **action**: a string that specifies the generic action to perform (such as *view* or *edit*)



Android Intents (cont.)

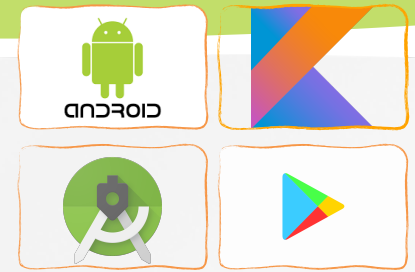
- » The primary information in an intent
 - **category** gives additional information about the action to execute
 - **type** specifies an explicit type (a MIME type) of the intent data.
 - **extras**: this is a [Bundle](#) of any additional information.



Android Intents (cont.)

» Some examples of Intent's action/data pairs:

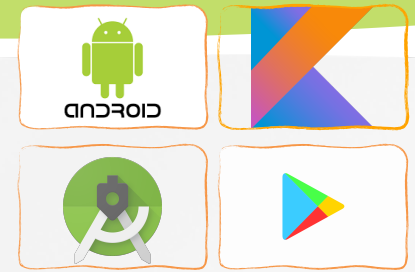
- **ACTION_VIEW** *content://contacts/people/1* -- Display information about the person whose identifier is "1".
- **ACTION_DIAL** *content://contacts/people/1* -- Display the phone dialer with the person filled in.
- **ACTION_VIEW** *tel:123* -- Display the phone dialer with the given number filled in. Note how the VIEW action does what is considered the most reasonable thing for a particular URI.
- **ACTION_DIAL** *tel:123* -- Display the phone dialer with the given number filled in.



Android Intents (cont.)

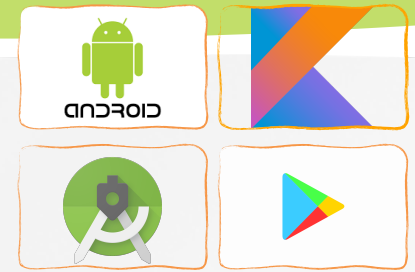
» Some examples of Intent's action/data pairs:

- **ACTION_EDIT** *content://contacts/people/1* -- Edit information about the person whose identifier is "1".
- **ACTION_VIEW** *content://contacts/people/* -- Display a list of people, which the user can browse through.
- This example is a typical top-level entry into the Contacts application, showing you the list of people. Selecting a particular person to view would result in a new intent { **ACTION_VIEW** *content://contacts/people/N* } being used to start an activity to display that person.



Intent Filter

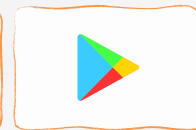
- » This is an expression in an app's manifest file that specifies the type of intents that the component would like to receive
 - `<action>` Declares the intent action accepted
 - `<data>` Declares the type of data accepted,
 - `<category>` Declares the intent category accepted



Intent Filter

» Example, here's an activity declaration with an intent filter to receive an **ACTION_SEND** intent when the data type is text

```
<activity android:name=".ShareActivity">  
  <intent-filter>  
    <action android:name="android.intent.action.SEND"/>  
    <category android:name="android.intent.category.DEFAULT"/>  
    <data android:mimeType="text/plain"/>  
  </intent-filter>  
</activity>
```

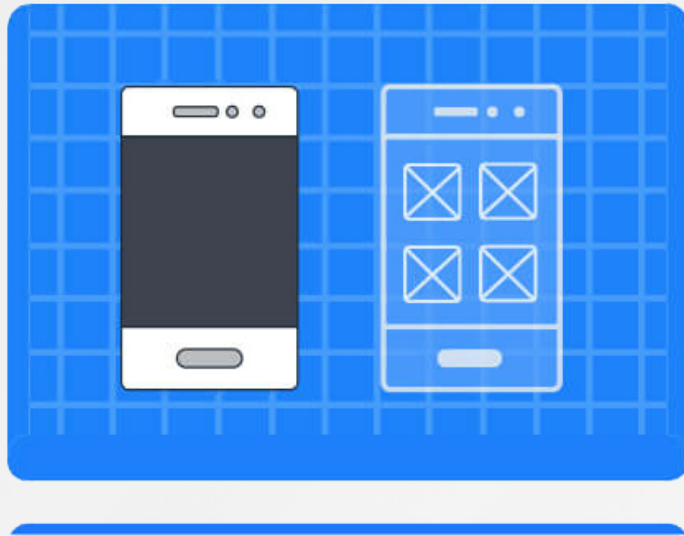
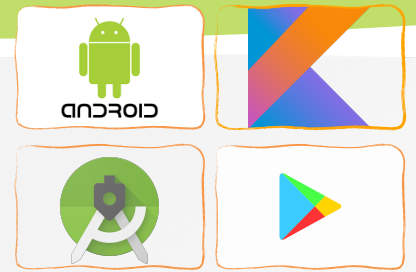


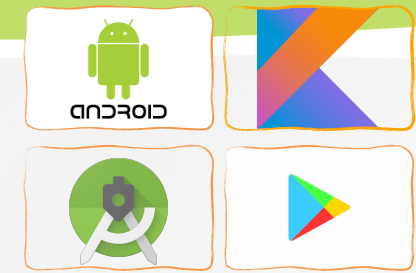
Intent Filter

```
<activity android:name="MainActivity">
  <!-- This activity is the main entry, should appear in app launcher -->
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>

<activity android:name="ShareActivity">
  <!-- This activity handles "SEND" actions with text data -->
  <intent-filter>
    <action android:name="android.intent.action.SEND"/>
    <category android:name="android.intent.category.DEFAULT"/>
    <data android:mimeType="text/plain"/>
  </intent-filter>
  <!-- This activity also handles "SEND" and "SEND_MULTIPLE" with media data -->
  <intent-filter>
    <action android:name="android.intent.action.SEND"/>
    <action android:name="android.intent.action.SEND_MULTIPLE"/>
    <category android:name="android.intent.category.DEFAULT"/>
    <data android:mimeType="application/vnd.google.panorama360+jpg"/>
    <data android:mimeType="image/*"/>
    <data android:mimeType="video/*"/>
  </intent-filter>
</activity>
```

Demo

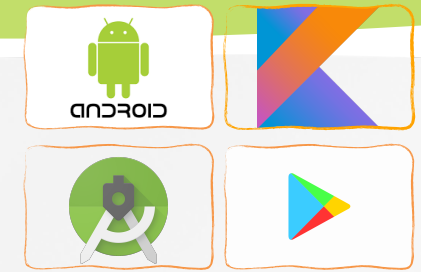




Example1: Start a second activity

Steps

- » File > New > Activity > Empty Views Activity.
- » Name the new Activity **SecondActivity**.
 - » Make sure **Generate Layout File** is checked.
 - » The layout name is filled in as `activity_second`.
 - » Do *not* check the **Launcher Activity** option.
- » **Finish.**
 - » New activity layout (`activity_second.xml`)
 - » New Java file (`SecondActivity.kt`)
 - » The `AndroidManifest.xml` file is updated to include the new Activity.

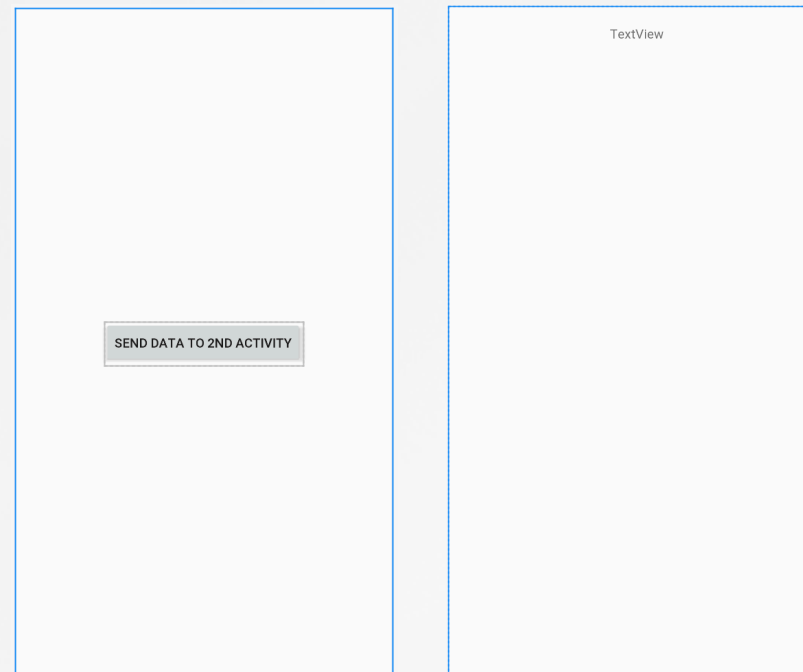


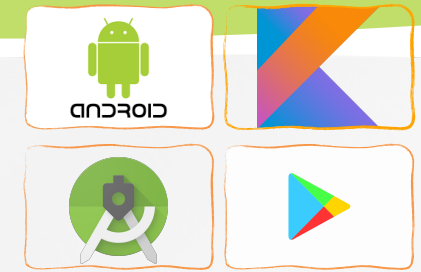
Example1: Start a second activity (cont.)

» Modify the AndroidManifest.xml file

```
<activity android:name=".SecondActivity"  
    android:label="@string/secondActivity"  
    android:parentActivityName=".MainActivity">  
</activity>
```

» Design UI





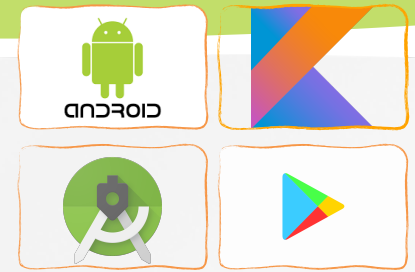
Example1: Start a second activity (cont.)

» Start 2nd activity

```
findViewById<Button>(R.id.gotoSecondBtn).setOnClickListener {  
    val intent = Intent(this, SecondActivity::class.java)  
    startActivity(intent);  
}
```

» Start with data

```
findViewById<Button>(R.id.gotoSecondBtn).setOnClickListener {  
    val intent = Intent(this, SecondActivity::class.java)  
  
    intent.putExtra(CONSTANTS.MESSAGE1, "This is value")  
  
    startActivity(intent);  
}
```

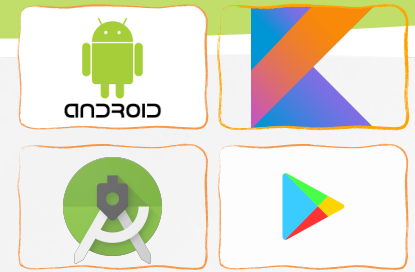


Example1: Start a second activity (cont.)

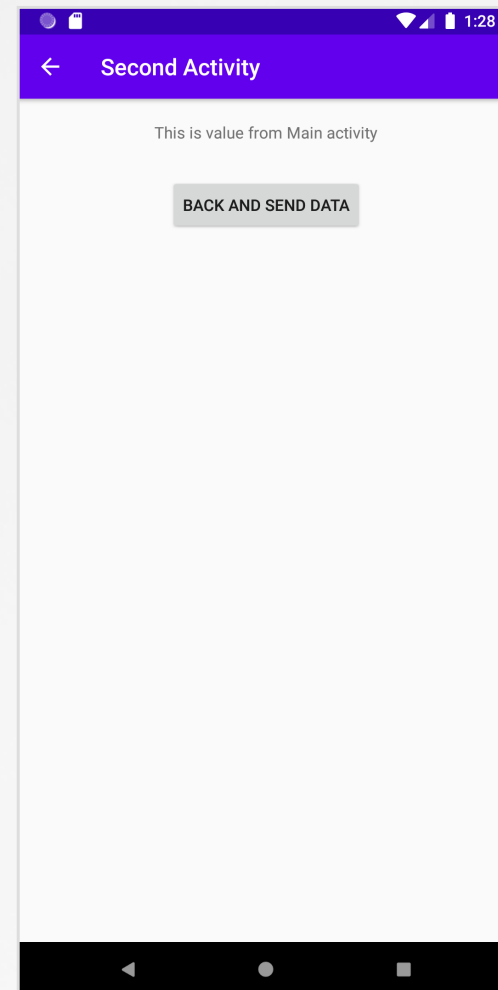
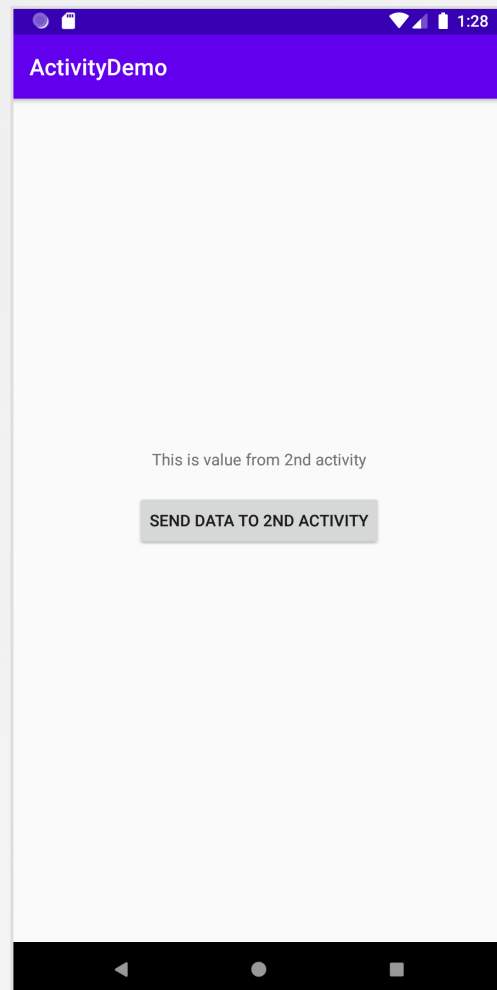
» Read data in 2nd activity

```
class SecondActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_second)  
  
        val intent = intent  
        val message = intent.getStringExtra(CONSTANTS.MESSAGE1)  
        val textView: TextView = findViewById(R.id.contentTv)  
        textView.text = message  
    }  
}
```

Example2: Return data back to the main Activity



» Modify UI

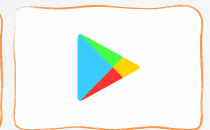




Example2: Return data back to the main Activity (cont.)

» Modify code in MainActivity

```
...  
findViewById<Button>(R.id.gotoSecondBtn).setOnClickListener {  
    val intent = Intent(this, SecondActivity::class.java)  
    intent.putExtra(CONSTANTS.MESSAGE1, "This is value from Main activity")  
  
    startActivityForResult(intent, REQUEST_CODE)  
}  
...  
  
val REQUEST_CODE = 1111  
override fun onActivityResult(requestCode: Int, resultCode: Int, data: Intent?) {  
    super.onActivityResult(requestCode, resultCode, data)  
  
    if (requestCode === REQUEST_CODE) {  
        if (resultCode === Activity.RESULT_OK) {  
            val reply = data!!.getStringExtra(CONSTANTS.MESSAGE2)  
  
            val textView: TextView = findViewById(R.id.contentTv)  
            textView.text = reply  
        }  
    }  
}
```



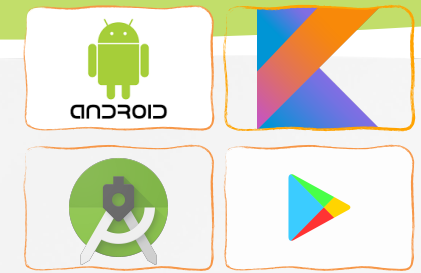
Example2: Return data back to the main Activity (cont.)

» Modify code in SecondActivity

```
val intent = intent
val message = intent.getStringExtra(CONSTANTS.MESSAGE1)
val textView: TextView = findViewById(R.id.contentTv)
textView.text = message

findViewById<Button>(R.id.backBtn).setOnClickListener {
    val reply: String = "This is value from 2nd activity"

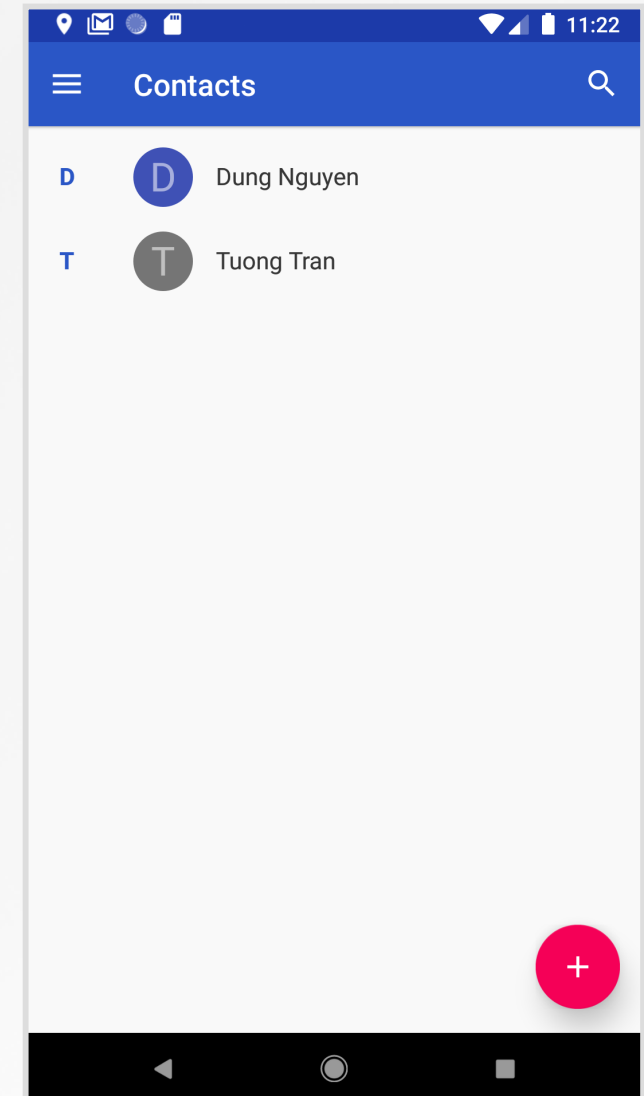
    val replyIntent = Intent()
    replyIntent.putExtra(CONSTANTS.MESSAGE2, reply)
    setResult(Activity.RESULT_OK, replyIntent)
    finish()
}
```

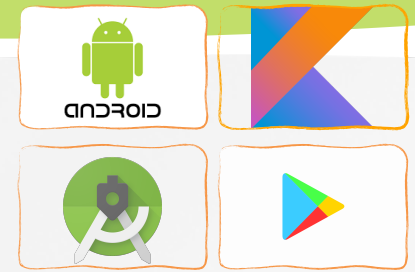


Example3: Built-in Intent

» The following code fragment calls an **Intent** whose job is to invoke a built-in task (**ACTION_VIEW**) and explore the *Contacts* available in the phone

```
val myIntent: Intent = Intent(Intent.ACTION_VIEW,  
    Uri.parse("content://contacts/people"))  
  
startActivity(myIntent)
```

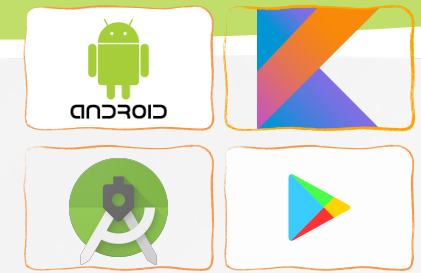




Example3: Built-in Intent

» The class including invocation to an Intent to display Contacts.

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val myIntent: Intent = Intent(Intent.ACTION_VIEW,  
            Uri.parse("content://contacts/people"))  
        startActivity(myIntent)  
    }  
}
```



Example4: implicit Intent

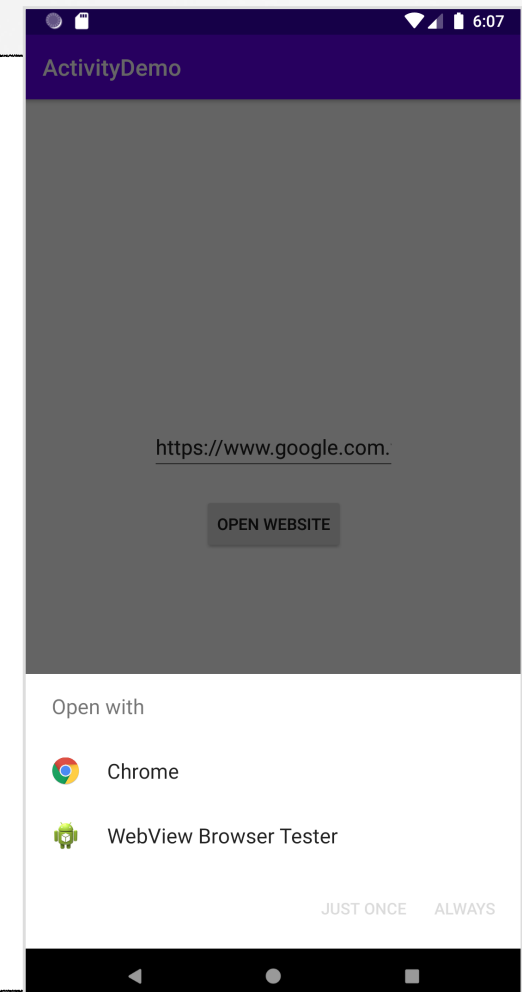
» Open url

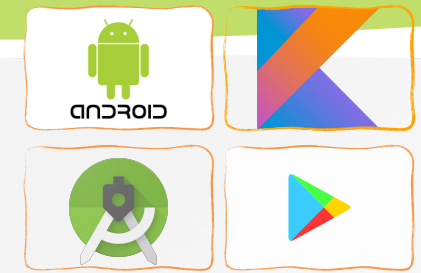
```
var contentEt: EditText? = null

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    contentEt = findViewById(R.id.urlEt)

    findViewById<Button>(R.id.openWebsiteBtn).setOnClickListener {
        try {
            // Get the URL text.
            val url = contentEt!!.text.toString()
            // Parse the URI and create the intent.
            val webpage: Uri = Uri.parse(url)
            val intent = Intent(Intent.ACTION_VIEW, webpage)
            startActivity(intent)
        }
        catch (e: Exception) {
            Log.i("ImplicitIntents", "Can't handle this intent!")
        }
    }
}
```

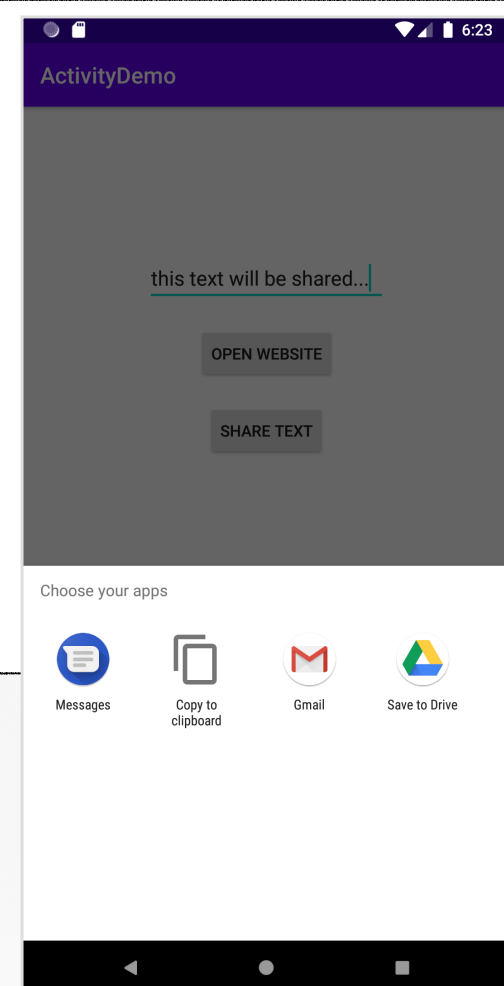


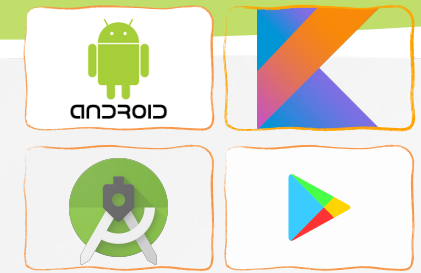


Example5: implicit Intent

» Share text

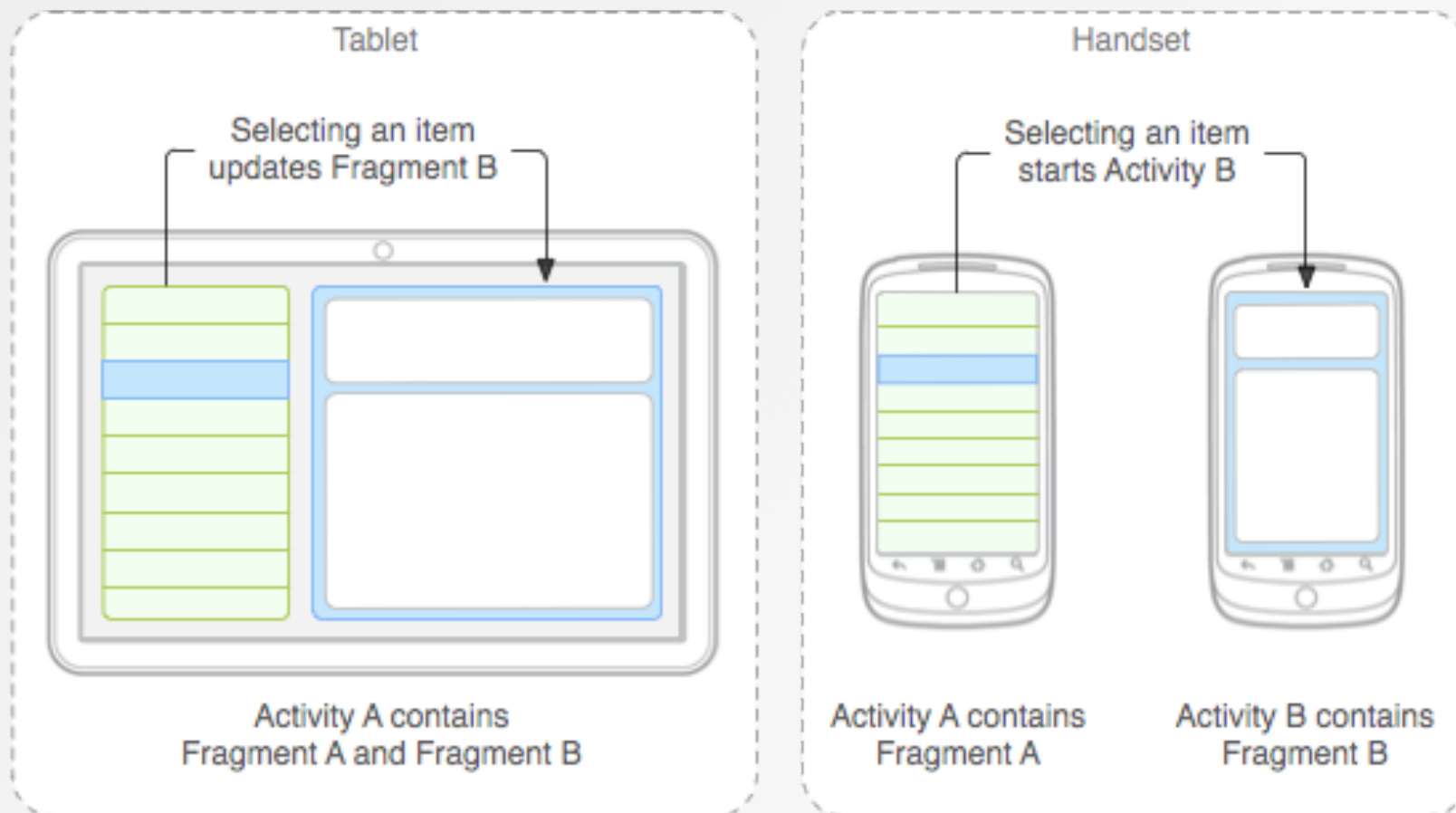
```
private fun shareText() {  
    findViewById<Button>(R.id.shareTextBtn).setOnClickListener {  
        val txt: String = contentEt!!.text.toString()  
  
        val mimeType = "text/plain"  
        ShareCompat.IntentBuilder  
            .from(this)  
            .setType(mimeType)  
            .setChooserTitle("Choose your apps")  
            .setText(txt)  
            .startChooser()  
    }  
}
```

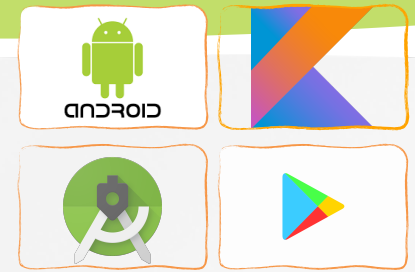




Fragments

- » A **Fragment** represents a behavior or a portion of user interface in a **FragmentActivity**.





Fragments

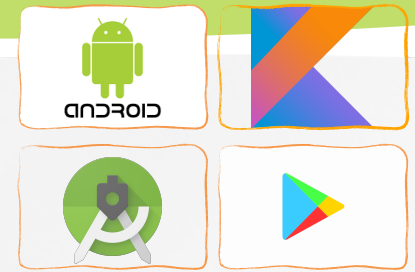
- » Fragment is an independent android component, used by an activity (like a sub-activity).
- » Fragment has its own life cycle and interface.
- » Fragments usually have a code file associated with the xml interface file.
 - » Fragments without an xml interface file are often referred to as headless fragments.
- » The fragment life cycle is directly affected by the life cycle of the host activity. For example, when the activity is paused, so are all segments within it, and when the activity is canceled, so are all segments.
- » A fragment can be used in many activity.
- » We use the `getActivity()` method to retrieve the parent Activity
- » The fragment is defined in the activity (static definition) xml file or the fragment can be modified at runtime (dynamic definition)
- » Fragment added to API 11 (android 3.0) and above.



- » User navigates backward or fragment is removed/replaced
- » The fragment is added to the back stack, then removed/replaced

» The fragment returns to the layout from the back stack





Common Lifecycle methods

» onCreate()

- » Creating the fragment
- » We should initialize essential components of the fragment that you want to retain when the fragment is paused or stopped, then resumed.

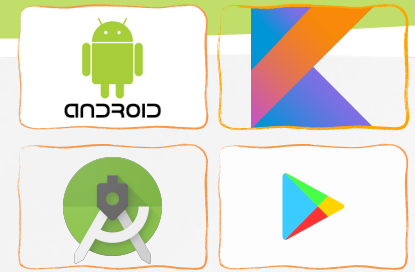
» onCreateView()

- » Drawing its user interface for the first time.
- » We must return a **View** from this method that is the root of your fragment's layout.
- » We can return null if the fragment does not provide a UI.

» onPause()

- » The system calls this method as the first indication that the user is leaving the fragment (though it doesn't always mean the fragment is being destroyed).
- » This is usually where you should commit any changes that should be persisted beyond the current user session (because the user might not come back)

Creating Fragments xml



» New / Fragment / Fragment (Blank)...

New Android Component

 Configure Component 

Creates a blank fragment that is compatible back to API level 4.

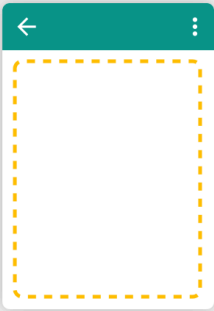
Fragment Name:

☒ Create layout XML?

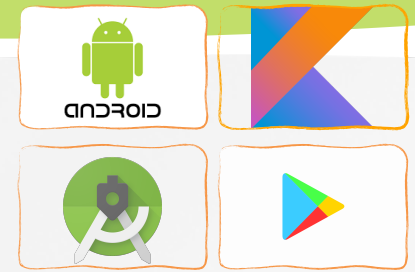
Fragment Layout Name:

☒ Include fragment factory methods?

Source Language:



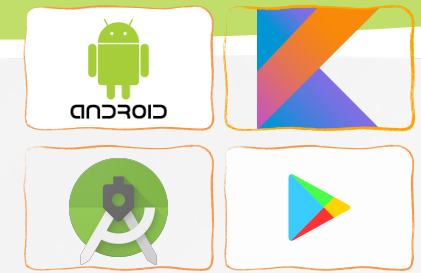
The name of the fragment class to create



Creating Fragments xml

» New / Fragment / Fragment (Blank)...

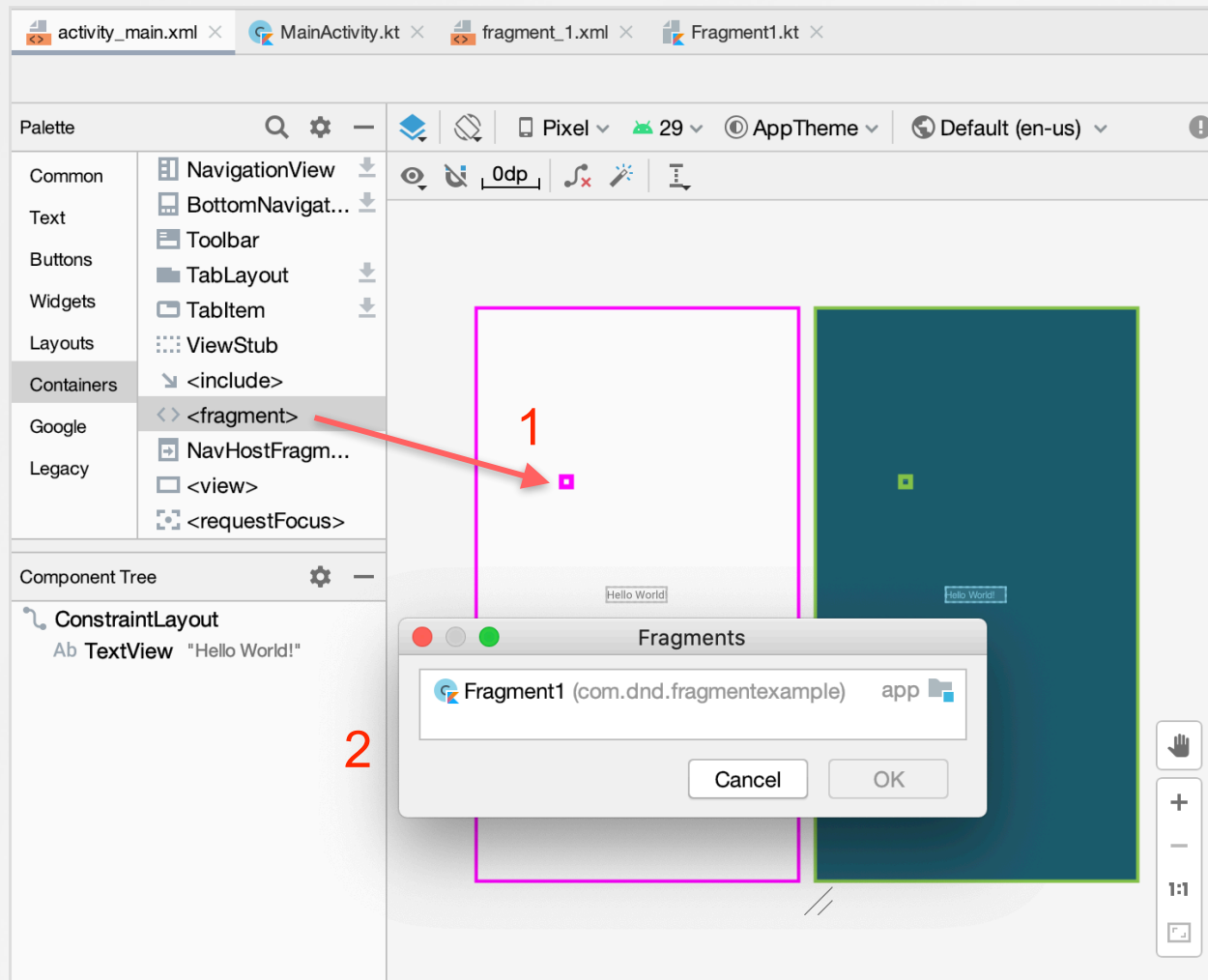
```
class Fragment1 : Fragment() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
    }  
  
    override fun onCreateView(  
        inflater: LayoutInflater, container: ViewGroup?,  
        savedInstanceState: Bundle?  
    ): View? {  
        // Inflate the layout for this fragment  
        return inflater.inflate(R.layout.fragment_1, container, false)  
    }  
}
```



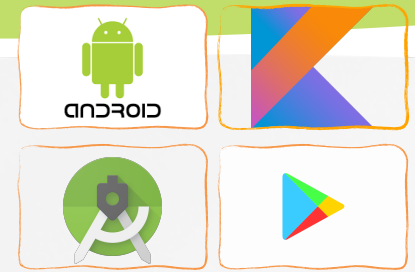
Creating Fragments (cont.)

xml

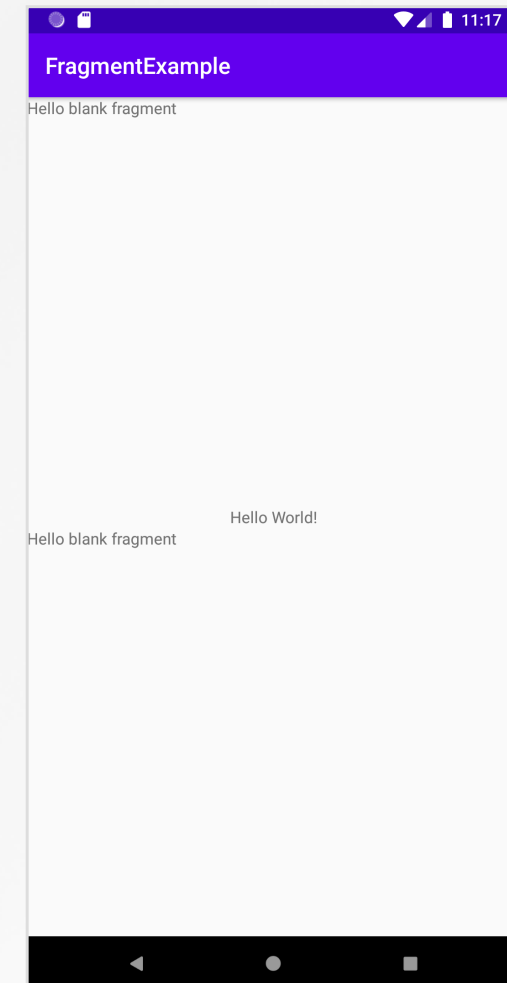
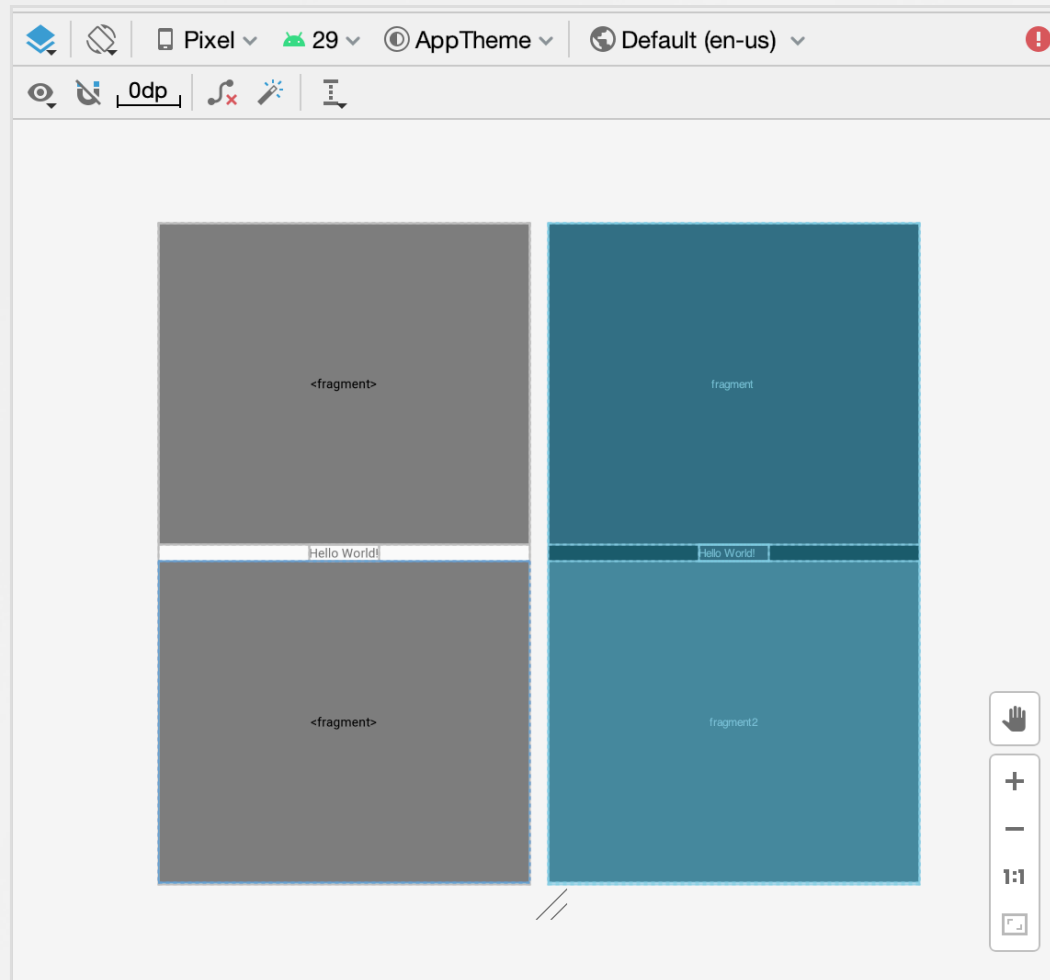
» Adding a fragment to an activity

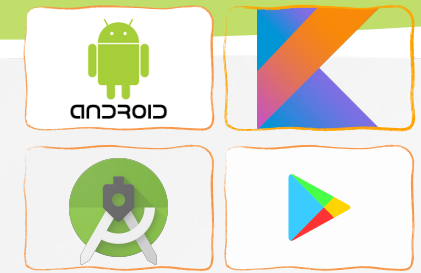


Creating Fragments (cont.) xml



» Adding a fragment to an activity (2) and Run





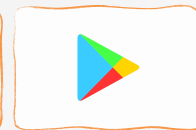
Creating Fragments Dynamic

- » Adding the fragment **dynamically** in Java using the **FragmentManager**
- » The **FragmentManager** class and the **FragmentTransaction** class allow us to add, remove and replace fragments in the layout of your activity at runtime

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="..."
    ...
    tools:context=".MainActivity">

    <fragment
        android:id="@+id/your_placeholder"
        android:name="com.dnd.fragmentexample.Fragment1"
        ...
        app:layout_constraintTop_toBottomOf="@+id/textView"
        tools:layout="@layout/fragment_1" />

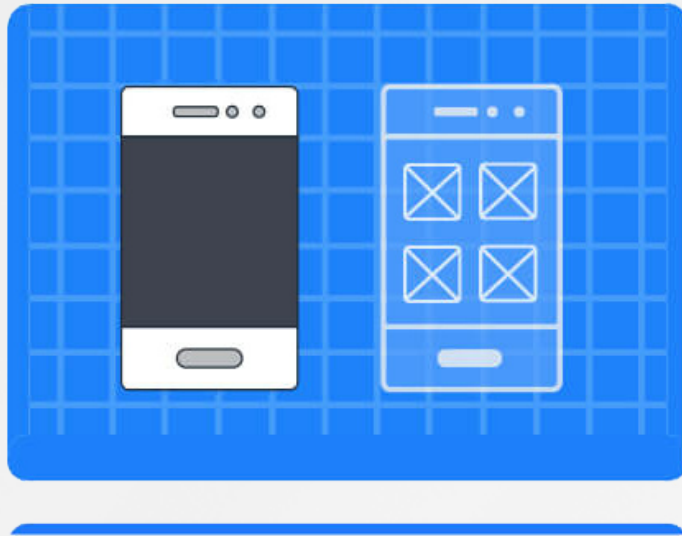
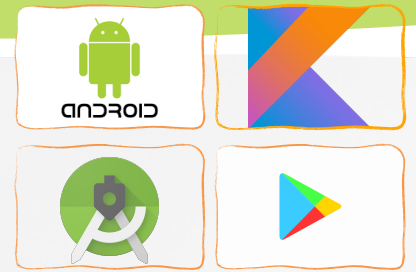
</androidx.constraintlayout.widget.ConstraintLayout>
```



Creating Fragments Dynamic

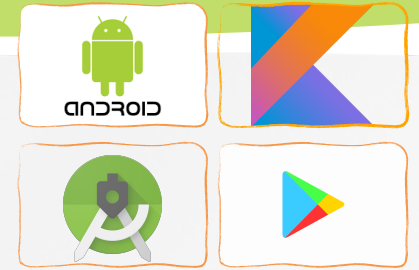
```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        // Begin the transaction  
        val ft: FragmentTransaction = supportFragmentManager.beginTransaction()  
  
        // Replace the contents of the container with the new fragment  
        ft.replace(R.id.your_placeholder, NewFragment())  
        // or ft.add(R.id.your_placeholder, new FooFragment());  
  
        ft.commit()  
    }  
}
```


Demo



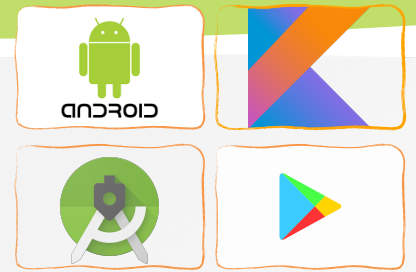
 **Kotlin**

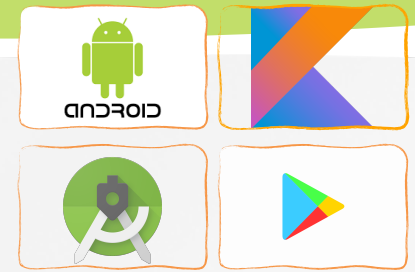
Example6: Using Fragments



» Example6-Using Fragments.zip

Q&A





Reference

- » <https://kotlinlang.org/docs/reference/>
- » Peter Späth, Learn Kotlin for Android Development, 2019
- » Ted Hagos, Learn Android Studio 3 with Kotlin, 2018
- » Peter Späth, Pro Android with Kotlin, 2019
- » Milos Vasic, Mastering Android Development with Kotlin, 2017
- » Victor Matos, Mobile Application Development course, Cleveland State University
- » <https://developer.android.com/guide/>